

# Unsupervised Text Clustering with K-means

Cesare Fraccaroli - VR533061

NLP Course Project

## Abstract

Three document embeddings are compared: **custom TF-IDF** (from scratch), **scikit-learn TF-IDF**, and **Gensim Word2Vec**, for **K-Means** on *Reuters*. Preprocessing and clustering are decoupled via CSV so stages can be re-run separately; tokenization, stopwords, stemming, and embedding are configurable flags. **Word2Vec** gives the best numbers (Silhouette 0.31, Davies-Bouldin 1.39). TF-IDF only gives usable clusters after TruncatedSVD.

## 1 Introduction

We look at how text representation choice affects **K-Means** on *Reuters-21578* (10,788 documents, 90 categories). Documents are *multi-labeled* (mean 1.24 categories each) and the distribution is *heavily imbalanced*: *earn* and *acq* have 3,000+ documents each. That makes unsupervised partitioning non-trivial.

## 2 Design Choices

### 2.1 Pipeline Architecture

The pipeline has two stages: preprocessing (run once) and clustering (re-run as needed).

1. **Preprocessing** (`preprocess.py`): raw text  $\rightarrow$  NLP steps  $\rightarrow$  embedding  $\rightarrow$  optional SVD  $\rightarrow$  vectors saved as CSV.
2. **Clustering** (`cluster.py`, `tune.py`): load CSV  $\rightarrow$  K-Means  $\rightarrow$  evaluation.

Vectors are written to CSV, so clustering can be re-run for different  $k$  or distances without recomputing embeddings. Split is **80/20** (train 8,631; test 2,157), fixed seed.

### 2.2 Configurable Preprocessing

Flags switch each step on or off:

- **-tokenize**: Penn Treebank tokenization via NLTK `word_tokenize`. Required for all embeddings.
- **-stopwords**: removes 179 English stopwords (NLTK corpus).
- **-stem**: Porter Stemmer collapses morphological variants (*running*, *runs*, *ran*  $\rightarrow$  *run*) and shrinks vocabulary. We preferred stemming over lemmatization for speed and simplicity; the downside is occasional over-stemming.

Preprocessing can be ablated without code changes.

### 2.3 Embedding Methods

The **-embedding** flag selects one of three strategies:

| Embedding                  | Dims       | Key property               |
|----------------------------|------------|----------------------------|
| <code>tfidf-custom</code>  | $\sim 39k$ | Sparse, from-scratch impl. |
| <code>tfidf-sklearn</code> | $\sim 21k$ | Sparse, built-in norms     |
| <code>word2vec</code>      | 100        | Dense, semantic            |

$$\text{TF}(w, d) = \frac{\text{count}(w, d)}{|d|}$$

$$\text{IDF}(w) = \log\left(\frac{N}{\text{df}(w)}\right)$$

$$\text{TF-IDF}(w, d) = \text{TF}(w, d) \times \text{IDF}(w)$$

where  $w$  is a word,  $d$  is a document,  $N$  is the total number of documents, and  $\text{df}(w)$  is the number of documents containing  $w$ .

Both TF-IDF variants give *high-dimensional sparse* vectors (most entries zero). Vocabulary size differs ( $\sim 39\text{k}$  vs.  $\sim 21\text{k}$ ) because of scikit-learn’s tokenization. **Word2Vec** instead uses the **L2**-normalized mean of word vectors: *dense*, fixed size, *semantic similarity* rather than *lexical overlap*.

## 2.4 Dimensionality Reduction (SVD)

In 20k to 39k dimensions, TF-IDF vectors hit the *curse of dimensionality*: pairwise **cosine distances** tend to look alike, and K-Means struggles. **TruncatedSVD** (same idea as *Latent Semantic Analysis*) reduces them to 300 components, keeping most of the useful variance and dropping noise from rare terms.

SVD is applied at preprocessing (`-svd 300`), fit on the *training set* and applied to both splits so clustering and evaluation use the same *vector space*. Applying SVD only inside the clusterer would leave evaluation in the original space and skew the metrics.

Word2Vec is already 100 *dense* dimensions; no SVD.

## 2.5 Clustering and k Selection

Clustering: NLTK `KMeansClusterer`, **cosine distance**, **L2** norm.  $k$  is chosen by the **elbow method**: **WCSS** vs  $k$ , pick the bend where adding clusters hardly helps.

## 3 Results

### 3.1 Elbow Analysis

WCSS for  $k = 1$  to 10 (Figures 1 to 3).

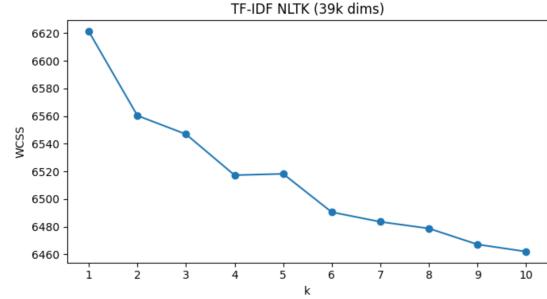


Figure 1: WCSS vs.  $k$ : Custom TF-IDF. Curve is nearly flat (about 2 to 3% variation).

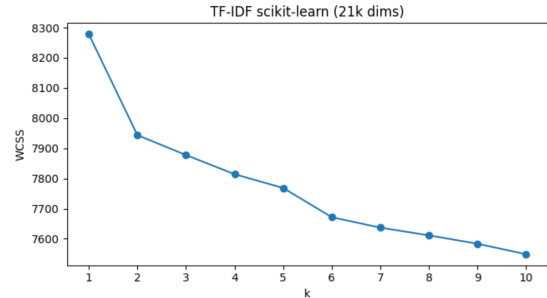


Figure 2: WCSS vs.  $k$ : TF-IDF sklearn. Curve is nearly flat.

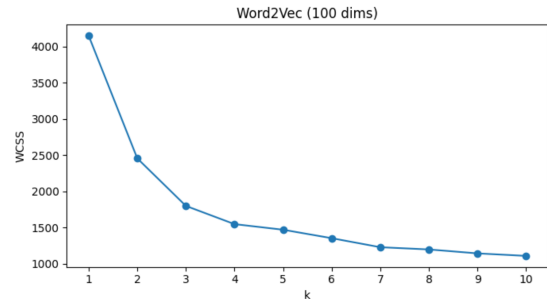


Figure 3: WCSS vs.  $k$ : Word2Vec. Clear elbow at  $k = 4$  to 5.

TF-IDF: curves almost flat, WCSS barely changes with  $k$ . In 20k to 39k *sparse* dimensions K-Means does not separate clusters well. **Word2Vec** has a clear elbow: WCSS 4,148 ( $k=1$ ) down to 1,547 ( $k=4$ ), then flat; about 4 to 5 groups. We set  $k=6$  for the rest.

## 3.2 Test Set Evaluation

Table 1: baseline (no SVD) vs SVD=300.

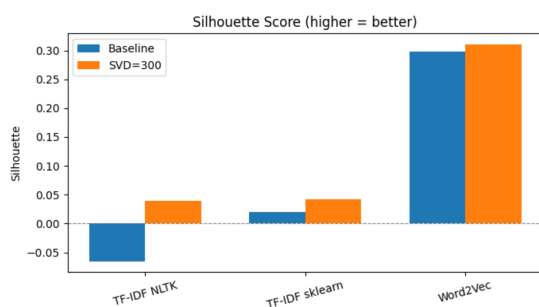


Figure 4: Silhouette score: baseline vs. SVD=300 across embeddings.

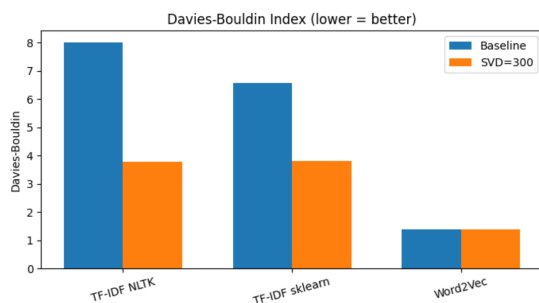


Figure 5: Davies-Bouldin index: baseline vs. SVD=300 across embeddings.

**Word2Vec**: Silhouette 0.31, Davies-Bouldin 1.39. Raw TF-IDF: Silhouette  $\approx 0$  (*near-random*). With SVD, TF-IDF improves (Silhouette about +0.04, Davies-Bouldin about halved) but stays well below Word2Vec.

## 3.3 Cluster Visualization

t-SNE of the test set, points colored by cluster (Figures 6 to 8).

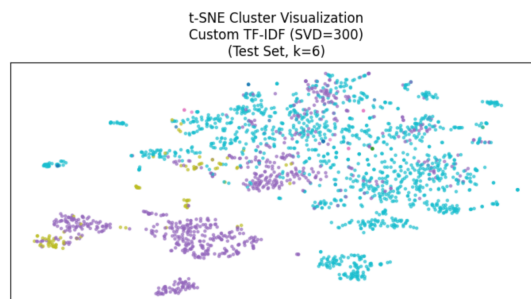


Figure 6: t-SNE projection: Custom TF-IDF (SVD=300),  $k=6$ . Heavy overlap.

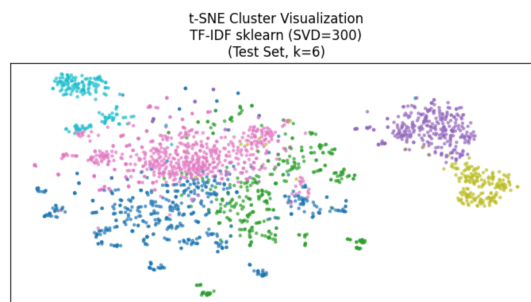


Figure 7: t-SNE projection: TF-IDF sklearn (SVD=300),  $k=6$ . Overlap; sklearn variant better than custom.

| Experiment | Embedding      | Dims   | WCSS     | Silhouette | Davies-Bouldin |
|------------|----------------|--------|----------|------------|----------------|
| Baseline   | Custom TF-IDF  | 39,204 | 907.04   | -0.0652    | 8.0253         |
| Baseline   | TF-IDF sklearn | 21,519 | 1,897.93 | 0.0202     | 6.5651         |
| Baseline   | Word2Vec       | 100    | 327.81   | 0.2986     | 1.3788         |
| SVD=300    | Custom TF-IDF  | 300    | 284.70   | 0.0396     | 3.7895         |
| SVD=300    | TF-IDF sklearn | 300    | 656.30   | 0.0427     | 3.8062         |
| -          | Word2Vec       | 100    | 322.24   | 0.3106     | 1.3923         |

Table 1: Test set evaluation ( $k = 6$ , cosine distance, 5 repeats). Silhouette ranges from  $-1$  to  $1$  (higher is better). Davies-Bouldin: lower is better.

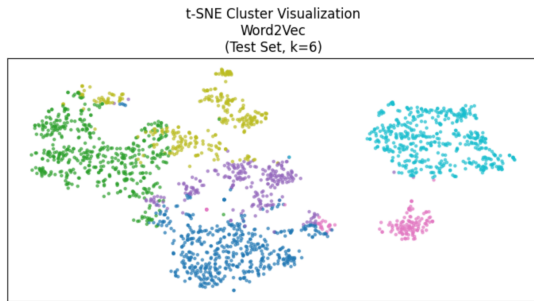


Figure 8: t-SNE projection: Word2Vec,  $k = 6$ . Clearly separated clusters.

**Word2Vec:** *spatially distinct* blobs, little overlap. The two TF-IDF plots are *interleaved*, boundaries unclear.

## 4 Conclusions

1. **Embedding.** *Word2Vec* (Silhouette 0.31) clearly beats both TF-IDF variants ( $\leq 0.04$ ), with or without SVD.
2. **SVD for TF-IDF.** In 20k to 39k *sparse* dimensions, *distance concentration* gives *near-random* assignments. SVD to 300 dims roughly halves *Davies-Bouldin* but does not match Word2Vec.
3. **Preprocessing flags.** Tokenization, stop-words, stemming, SVD as separate flags allow ablation. CSV between preprocessing and clustering avoids re-running embeddings.
4. **Limitations.** Reuters is *multi-label*, K-Means is *hard partitioning*; the two do not align. Word2Vec was not tuned. **HDB-SCAN** or **LDA** may fit this setting better. Also, trying different kinds of embeddings (e.g. **GloVe** or **BERT**) could be a future possible test.